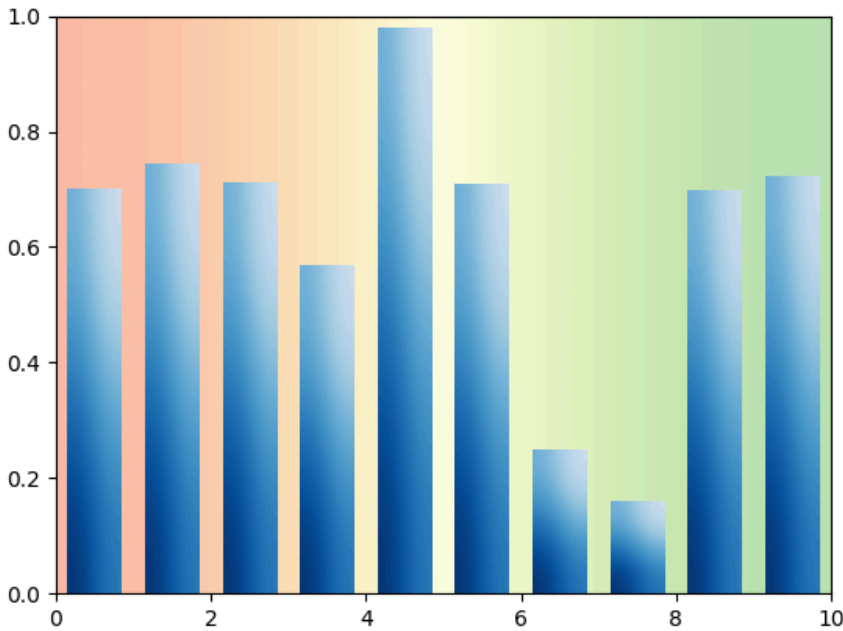


Bar chart with gradients

Matplotlib does not natively support gradients. However, we can emulate a gradient-filled rectangle by an `AxesImage` of the right size and coloring.

In particular, we use a colormap to generate the actual colors. It is then sufficient to define the underlying values on the corners of the image and let bicubic interpolation fill out the area. We define the gradient direction by a unit vector v . The values at the corners are then obtained by the lengths of the projections of the corner vectors on v .

A similar approach can be used to create a gradient background for an Axes. In that case, it is helpful to use Axes coordinates (`extent=(0, 1, 0, 1)`, `transform=ax.transAxes`) to be independent of the data coordinates.



```
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(19680801)

def gradient_image(ax, direction=0.3, cmap_range=(0, 1), **kwargs):
    """
    Draw a gradient image based on a colormap.

    Parameters
    -----
    ax : Axes
        The Axes to draw on.
    direction : float
        The direction of the gradient. This is a number in
        range 0 (=vertical) to 1 (=horizontal).
    cmap_range : float, float
        The fraction (cmin, cmay) of the colormap that should be
        used for the gradient, where the complete colormap is (0, 1).
    **kwargs
        Other parameters are passed on to `Axes.imshow()`.
        In particular, *cmap*, *extent*, and *transform* may be useful.
    """
    phi = direction * np.pi / 2
    v = np.array([np.cos(phi), np.sin(phi)])
    X = np.array([v @ [1, 0], v @ [1, 1],
                  [v @ [0, 0], v @ [0, 1]]])
    a, b = cmap_range
    X = a + (b - a) / X.max() * X
    im = ax.imshow(X, interpolation='bicubic', clim=(0, 1),
                   aspect='auto', **kwargs)
    return im

def gradient_bar(ax, x, y, width=0.5, bottom=0):
    for left, top in zip(x, y):
        right = left + width
        gradient_image(ax, extent=(left, right, bottom, top),
                      cmap=plt.cm.Blues_r, cmap_range=(0, 0.8))

fig, ax = plt.subplots()
ax.set(xlim=(0, 10), ylim=(0, 1))

# background image
gradient_image(ax, direction=1, extent=(0, 1, 0, 1), transform=ax.transAxes,
              cmap=plt.cm.RdYlGn, cmap_range=(0.2, 0.8), alpha=0.5)

N = 10
x = np.arange(N) + 0.15
y = np.random.rand(N)
gradient_bar(ax, x, y, width=0.7)
plt.show()
```

[Download Jupyter notebook: gradient_bar.ipynb](#)

[Download Python source code: gradient_bar.py](#)

- Capstyle
- Plotting categorical variables
- Plotting the coherence of two signals
- Cross spectral density (CSD)
- Curve with error band
- Errorbar limit selection
- Errorbar subsampling
- EventCollection Demo
- Eventplot demo
- Filled polygon
- Fill Between and Alpha
- Filling the area between lines
- Fill Betweenx Demo
- Hatch-filled histograms
- Bar chart with gradients**
- Hat graph
- Discrete distribution as horizontal bar chart
- JoinStyle
- Customizing dashed line styles
- Lines with a ticked patheffect
- Linestyles
- Marker reference
- Markevery Demo
- Plotting masked and NaN values
- Multicolored lines
- Mapping marker properties to multivariate data
- Power spectral density (PSD)
- Scatter Demo2
- Scatter plot with histograms

